



ECCV 2026 Tutorial

Aligning Diffusion Models with Human Preferences

Advantage Weighted Matching

From reward gradients to denoising targets

Shuchen Xue Research Scientist @  **NVIDIA** Efficient AI Team & Singapore Lab

Based on joint work with Chongjian Ge, Shilong Zhang, Yichen Li and Zhi-Ming Ma.

arxiv.org/abs/2509.25050

A good-looking image can still be misaligned

Three purple foxes
below one blue bird



A zebra and a chair



A runway displaying
“Spring Collection 2024”



Before alignment

How can a reward on the final image train the generator?

Reward optimization for diffusion models



$$\max_{\theta} J(\theta) = \mathbb{E}_{c \sim \mu} \mathbb{E}_{x \sim p_{\theta}(\cdot | c)} [R(x, c)].$$

General conditioning

Text, reference images, reference videos, or other reference data.

Reward on the final output

The model and sampler define $p_{\theta}(x | c)$.
 $R(x, c)$ evaluates the final generated output.

Policy gradient: differentiate the density

$$J(\theta) = \mathbb{E}_{c \sim \mu} \mathbb{E}_{x \sim p_\theta(\cdot | c)} [R(x, c)].$$

Assume the output log likelihood and its parameter gradient are available.

$$\begin{aligned} \nabla_\theta J(\theta) &= \mathbb{E}_c \int R(x, c) \nabla_\theta p_\theta(x | c) \, \mathrm{d}x \\ &= \mathbb{E}_c \int R(x, c) p_\theta(x | c) \nabla_\theta \log p_\theta(x | c) \, \mathrm{d}x \\ &= \mathbb{E}_{c,x} [R(x, c) \nabla_\theta \log p_\theta(x | c)]. \end{aligned}$$

Since $\mathbb{E}_{x|c} [\nabla_\theta \log p_\theta(x | c)] = 0$, a baseline preserves the mean:

$$\nabla_\theta J = \mathbb{E}_{c,x} [(R(x, c) - b(c)) \nabla_\theta \log p_\theta(x | c)].$$

Only reward values are needed.

Pathwise gradient: differentiate the sample

Assume a differentiable sampling map and a reward differentiable with respect to x .

$$x = g_{\theta}(z, c), \quad z \sim p(z), \quad J(\theta) = \mathbb{E}_{c,z}[R(g_{\theta}(z, c), c)].$$

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \mathbb{E}_{c,z}[R(g_{\theta}(z, c), c)] \\ &= \mathbb{E}_{c,z}[\nabla_{\theta} R(g_{\theta}(z, c), c)] \\ &= \mathbb{E}_{c,z}[D_{\theta} g_{\theta}(z, c)^{\top} \nabla_x R(g_{\theta}(z, c), c)] . \end{aligned}$$

Apply the chain rule through the generator and the reward.

No output likelihood is needed; the reward must be differentiable with respect to x .

Same expected gradient, different sample estimates

Suppose both routes are valid for the same distribution, with $x = g_\theta(z, c)$.

Policy-gradient estimate

$$G_{\text{PG}}(c, z) = (R(x, c) - b(c)) \nabla_\theta \log p_\theta(x \mid c), \quad x = g_\theta(z, c).$$

Pathwise estimate

$$G_{\text{PW}}(c, z) = D_\theta g_\theta(z, c)^\top \nabla_x R(g_\theta(z, c), c).$$

$$\mathbb{E}_{c,z}[G_{\text{PG}}] = \mathbb{E}_{c,z}[G_{\text{PW}}] = \nabla_\theta J.$$

The estimates generally differ even for the same generated sample.

Likelihood and differentiable sampling

A generator may provide a likelihood, a differentiable sampling map, or both.

Generator	Output likelihood	Backprop through sampling
GAN (implicit)	Usually not directly available	Yes, for a differentiable generator
Discrete AR	Tractable: $\prod_i p_\theta(x_i \mid x_{<i}, c)$	No direct pathwise gradient through tokens
Diffusion (continuous)	ODE likelihood: possible but costly; SDE: ELBO bound.	Yes, but backpropagating through many steps is costly.

Implicit generator: $x = g_\theta(z, c)$ defines a distribution by sampling, without a directly evaluable density.

Discrete AR models can backpropagate through log likelihood, but not directly through sampled tokens.

Two kinds of reward feedback

How good is x ?

$$R(x, c)$$

Zeroth-order reward feedback

No gradient through the reward is required.

Human ratings, rule-based checks, or VLM-as-a-judge with chain-of-thought reasoning.

Which change helps?

$$\nabla_x R(x, c)$$

First-order reward feedback

The reward is differentiable with respect to x .

Neural aesthetic or image–text similarity scores that allow backpropagation to x .

KL-regularized reward maximization

$$J_{\beta}(\theta) = \mathbb{E}_c \left[\mathbb{E}_{p_{\theta}(\cdot | c)} R(x, c) - \beta D_{\text{KL}}(p_{\theta}(\cdot | c) \| p_{\text{ref}}(\cdot | c)) \right].$$

$$p^*(x | c) = \frac{p_{\text{ref}}(x | c) e^{R(x, c)/\beta}}{Z(c)}$$

p_{ref} : fixed reference distribution.

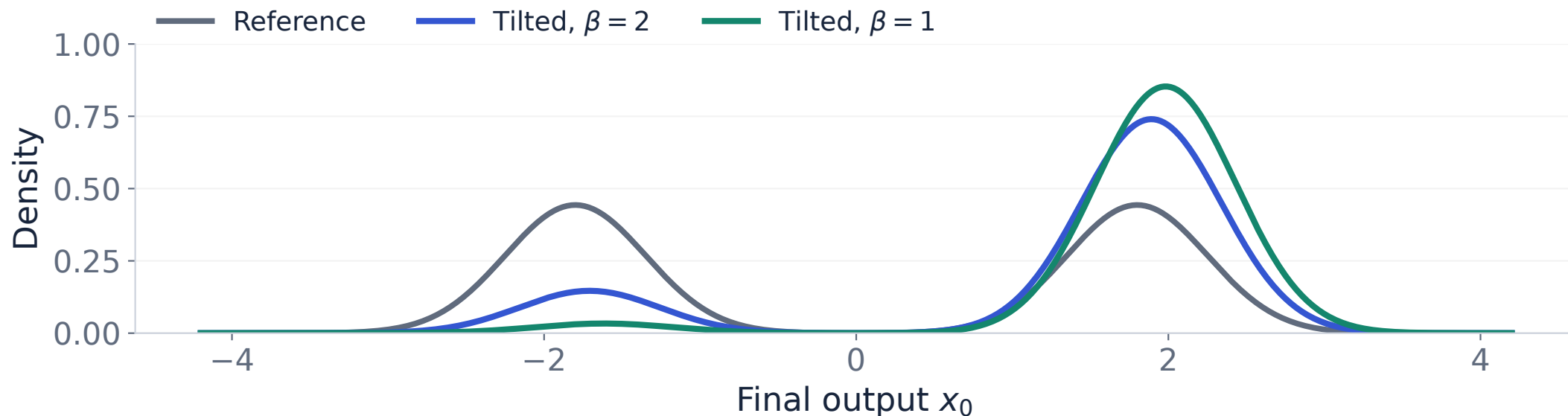
$\beta > 0$: KL regularization strength.

$$Z(c) = \mathbb{E}_{p_{\text{ref}}(\cdot | c)} e^{R(x, c)/\beta}.$$

Normalization constant.

$$J_{\beta} = \beta \mathbb{E}_c \log Z(c) - \beta \mathbb{E}_c D_{\text{KL}}(p_{\theta}(\cdot | c) \| p^*(\cdot | c)).$$

Higher reward means more probability mass

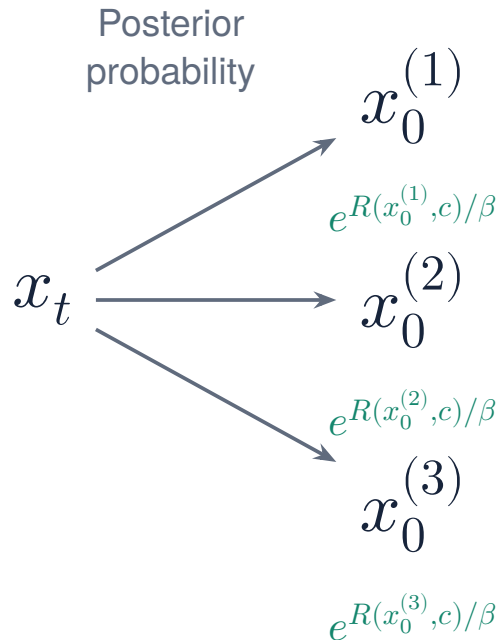


$R(x_0, c) = 0.9x_0$: smaller β favors the right-hand region more strongly.

Reward weighting at intermediate noise levels

The clean target gives high-reward outcomes more probability:

$$p_0^*(x_0 | c) = \frac{p_{\text{ref},0}(x_0 | c) e^{R(x_0, c)/\beta}}{Z(c)}.$$



Apply the same forward noising kernel $q_t(x_t | x_0)$:

$$\begin{aligned} p_t^*(x_t | c) &= \int q_t(x_t | x_0) p_0^*(x_0 | c) \mathrm{d}x_0 \\ &= \frac{p_{\text{ref},t}(x_t | c)}{Z(c)} \underbrace{\int p_{\text{ref}}(x_0 | x_t, c) e^{R(x_0, c)/\beta} \mathrm{d}x_0}_{h_t(x_t, c)}. \end{aligned}$$

Average the reward weight $e^{R/\beta}$ over possible clean outcomes.

The ideal score correction

$$\begin{aligned} p_t^*(x_t | c) &= \int q_t(x_t | x_0) p_0^*(x_0 | c) \mathrm{d}x_0 \\ &= \frac{p_{\text{ref},t}(x_t | c)}{Z(c)} \int p_{\text{ref}}(x_0 | x_t, c) e^{R(x_0, c)/\beta} \mathrm{d}x_0. \end{aligned}$$

Define the posterior average:

$$h_t(x_t, c) := \int p_{\text{ref}}(x_0 | x_t, c) e^{R(x_0, c)/\beta} \mathrm{d}x_0 = \mathbb{E}_{\text{ref}}[e^{R(x_0, c)/\beta} | x_t, c].$$

Then take the log and differentiate with respect to x_t :

$$\log p_t^*(x_t | c) = \log p_{\text{ref},t}(x_t | c) + \log h_t(x_t, c) - \log Z(c).$$

$$s_t^*(x_t, c) = s_{\text{ref},t}(x_t, c) + \nabla_{x_t} \log h_t(x_t, c).$$

$Z(c)$ is constant with respect to x_t .

The ideal correction is difficult to compute:

it requires a posterior expectation and its gradient with respect to x_t .

Reward optimization through log-likelihood

$$J_\beta(\theta) = \mathbb{E}_c \left[\mathbb{E}_{p_\theta(\cdot|c)} R(x_0, c) - \beta D_{\text{KL}}(p_\theta(\cdot | c) \| p_{\text{ref}}(\cdot | c)) \right].$$

We first derive the reward update.

$$J_R(\theta) = \mathbb{E}_{c, x_0 \sim p_\theta(\cdot|c)} R(x_0, c),$$

$$\nabla_\theta J_R(\theta) = \mathbb{E}_{c, x_0 \sim p_\theta(\cdot|c)} \left[\underbrace{(R(x_0, c) - b(c))}_{A(x_0, c)} \nabla_\theta \log p_\theta(x_0 | c) \right].$$

Consider on-policy update: $p_{\text{old}} = p_{\text{stopgrad}(\theta)}$.

$$\mathcal{L}_{\text{PG}}(\theta) = -\mathbb{E}_{c, x_0 \sim p_{\text{old}}(\cdot|c)} [\text{sg}[A(x_0, c)] \log p_\theta(x_0 | c)].$$

$$-\nabla_\theta \mathcal{L}_{\text{PG}}(\theta) = \nabla_\theta J_R(\theta).$$

How do we evaluate the log-likelihood of a diffusion model?

The diffusion ELBO gives a matching loss

The forward process gives a tractable lower bound on log-likelihood:

$$\log p_{\theta}(x_0 \mid c) \geq \text{ELBO}_{\theta}(x_0, c).$$

For a linear Gaussian forward diffusion,

$$x_t = (1 - t)x_0 + t\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad w_{\text{ELBO}}(t) = \frac{1 - t}{t}.$$

Its negative ELBO contains the familiar matching loss:

$$\begin{aligned} -\text{ELBO}_{\theta}(x_0, c) = & \underbrace{\mathbb{E}_{t, \epsilon} \left[w_{\text{ELBO}}(t) \|v_{\theta}(x_t, t, c) - (\epsilon - x_0)\|^2 \right]}_{\mathcal{L}_{\text{match}}^{\text{ELBO}}(\theta; x_0, c)} \\ & + \text{boundary terms} + \theta\text{-independent terms.} \end{aligned}$$

Use the diffusion ELBO as a tractable likelihood surrogate.

Advantage Weighted Matching

Replace the negative log-likelihood term with a matching loss:

$$\mathcal{L}_{\text{AWM}}(\theta) = \mathbb{E}_{c, x_0 \sim p_{\text{old}}, t, \epsilon} \left[\text{sg}[A(x_0, c)] \frac{1-t}{t} \|v_{\theta}(x_t, t, c) - (\epsilon - x_0)\|^2 \right].$$

A comes from the reward gradient; matching comes from the ELBO.

The ELBO specifies $w_{\text{ELBO}}(t) = \frac{1-t}{t}$ for uniform time sampling.

$$A > 0$$

Move **toward** the matching target.

$$A < 0$$

Move **away from** the matching target.

KL regularization as a velocity penalty

Apply the same forward SDE to p_θ and p_{ref} . Define

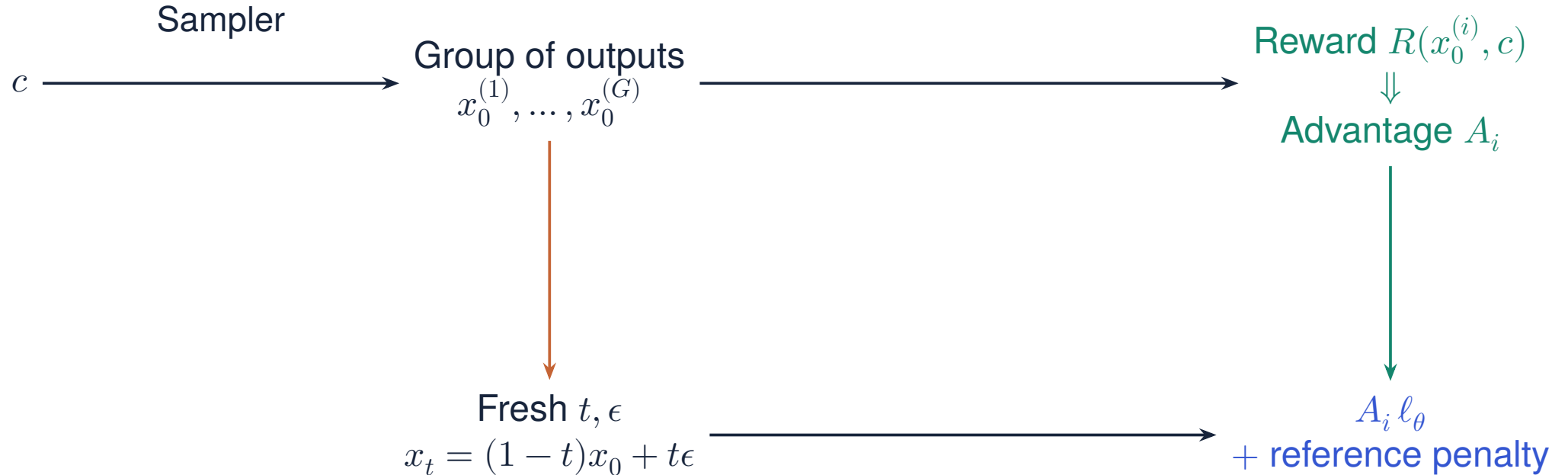
$$K_0(c) = D_{\text{KL}}(p_\theta(\cdot \mid c) \parallel p_{\text{ref}}(\cdot \mid c)).$$

For linear noising with a common noise endpoint,

$$\mathbb{E}_c K_0(c) = \mathbb{E}_{c, x_0 \sim p_{\theta, t, \epsilon}} \left[\frac{1-t}{t} \|v_{\theta, t}(x_t, c) - v_{\text{ref}, t}(x_t, c)\|^2 \right].$$

Song et al., Maximum Likelihood Training of Score-Based Diffusion Models (2021), Theorem 2 / Eq. (10).

AWM training procedure



Sampling solver can be chosen arbitrarily.

Likelihood assigns a specific weight to each time

Use the linear Markov diffusion with $q_t(x_t \mid x_0, c) = \mathcal{N}((1 - t)x_0, t^2 I)$.

With uniform time sampling, its negative-ELBO denoising loss is

$$\mathcal{L}_{\text{ELBO}}^{\text{interior}}(\theta; x_0, c) = \mathbb{E}_{t, \epsilon} \left[\frac{1 - t}{t} \|v_{\theta}(x_t, t, c) - (\epsilon - x_0)\|^2 \right].$$

A likelihood-derived choice

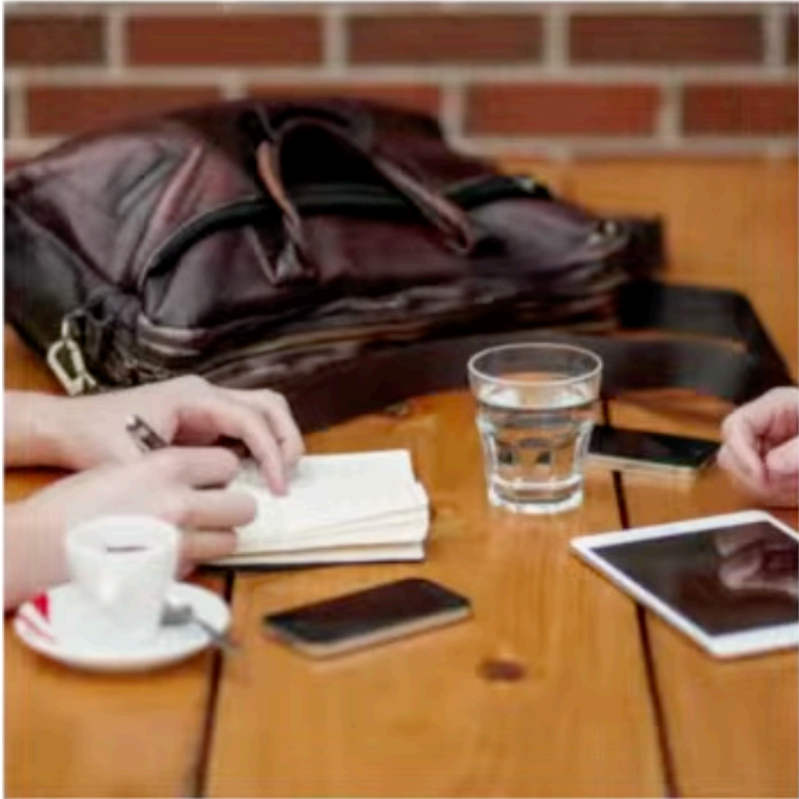
It follows from the diffusion path likelihood and the score-to-velocity conversion.

Does it match visual importance?

Small t receives large weight. That region often concerns information that is already almost visible.

Ho et al., DDPM (2020); Song et al., Maximum Likelihood Training of Score-Based Diffusion Models (2021); Kingma and Gao, Understanding Diffusion Objectives (2023).

Human perception does not value every bit equally



8-bit data



5-bit data

A substantial change in pixel representation can have a small perceptual effect.

Ruiqi Gao lecture, screenshots supplied by the speaker. Bit-depth quantization is distinct from spatial low-pass filtering.

Perceptual and semantic compression

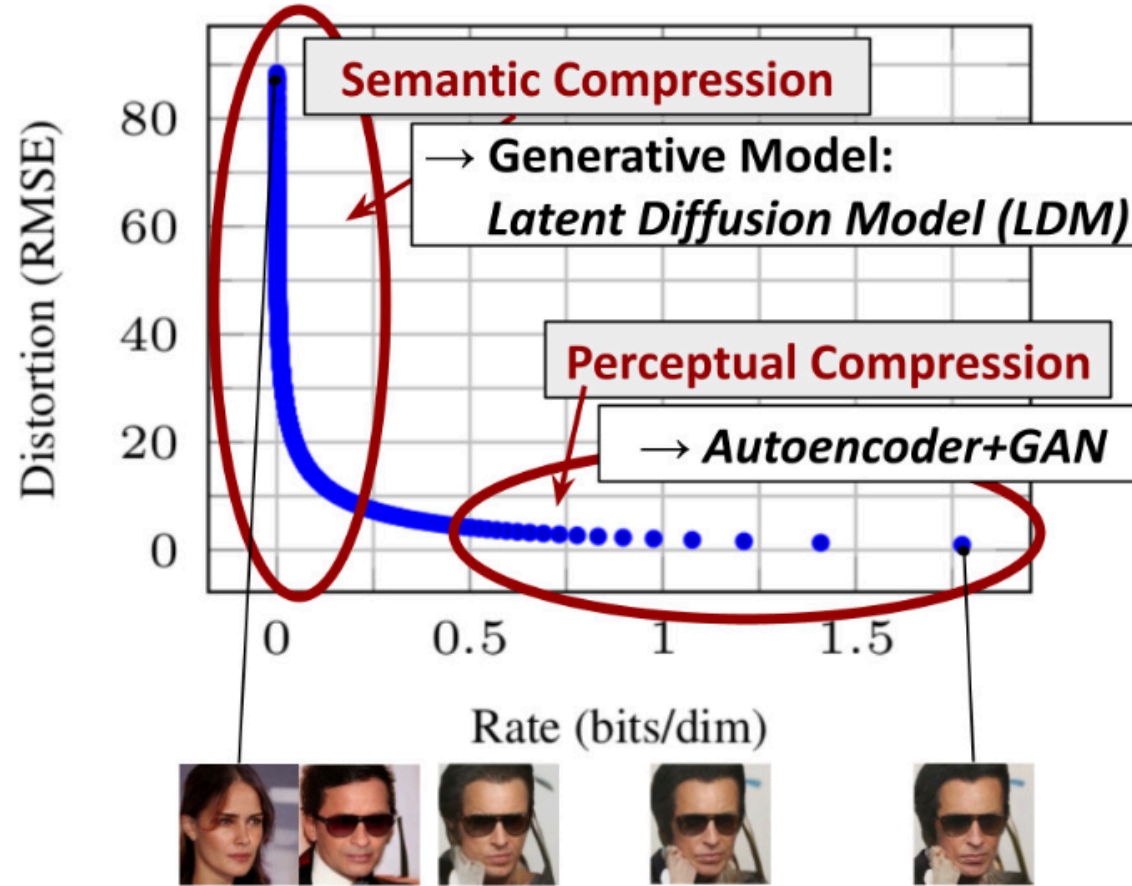
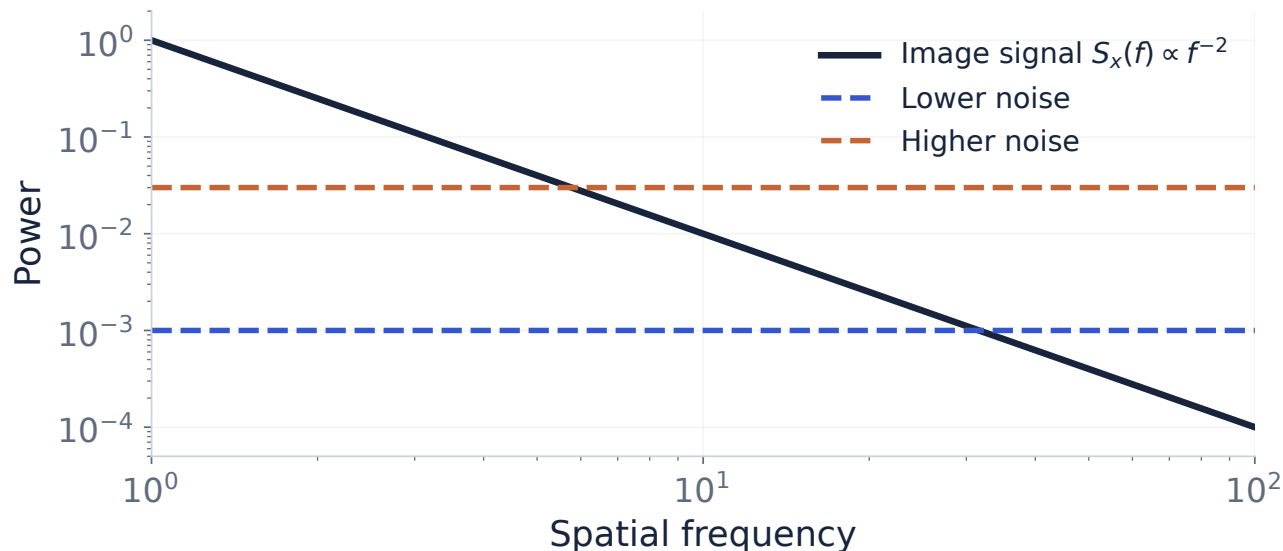


Image statistics create a coarse-to-fine order



Natural images often have more power at lower spatial frequencies.

White noise has approximately flat power. High-frequency signal is buried sooner.

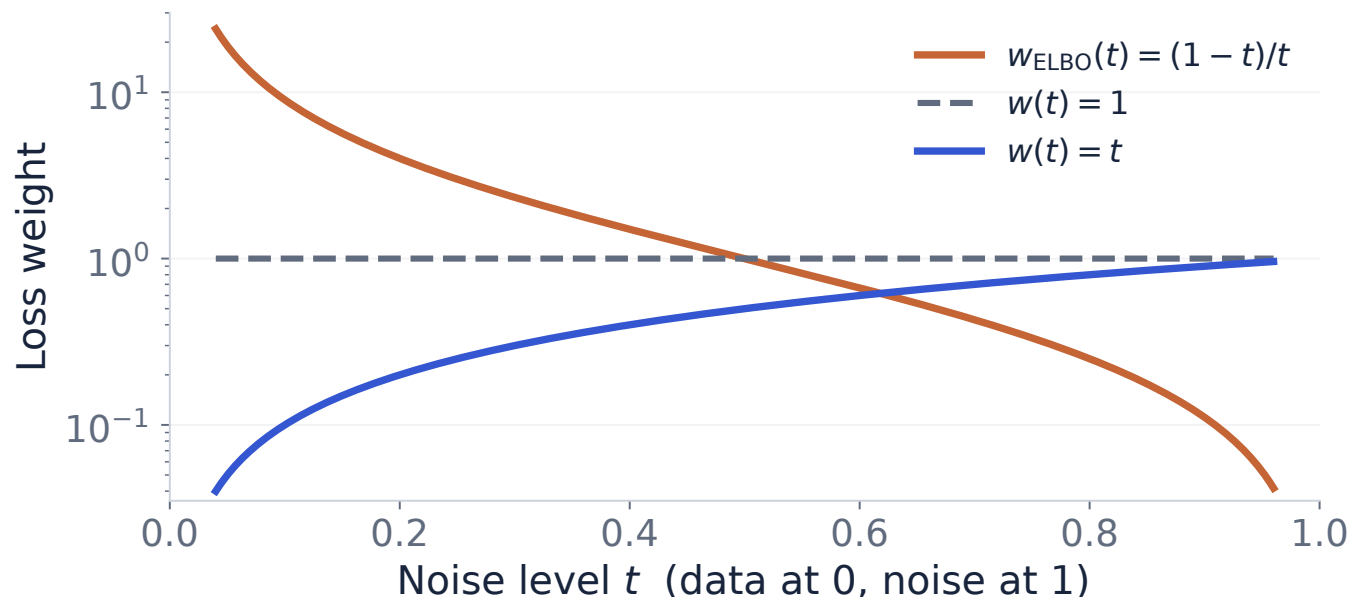
Reverse generation

Coarse structure tends to emerge before fine detail.

$$\text{SNR}(f, t) = \alpha_t^2 S_x(f) / \sigma_t^2.$$

Illustrative $S_x(f) \propto f^{-2}$. Dieleman, Diffusion is spectral autoregression (2024); Ruiqi Gao lecture.

The time weight is also a modeling decision



ELBO: $(1 - t)/t$

Emphasizes low-noise reconstruction.

AWM main experiments: t

Shifts training emphasis toward noisier states.

Match the training emphasis to the task and model capacity.

Uniform time sampling. Xue et al., AWM (2026), Section 4; Kingma and Gao, Understanding Diffusion Objectives (2023). Perception motivates reweighting; it does not prove $w(t) = t$ is optimal.

FLUX before and after AWM

Before

Three purple foxes
below one blue bird



After
100 AWM
updates



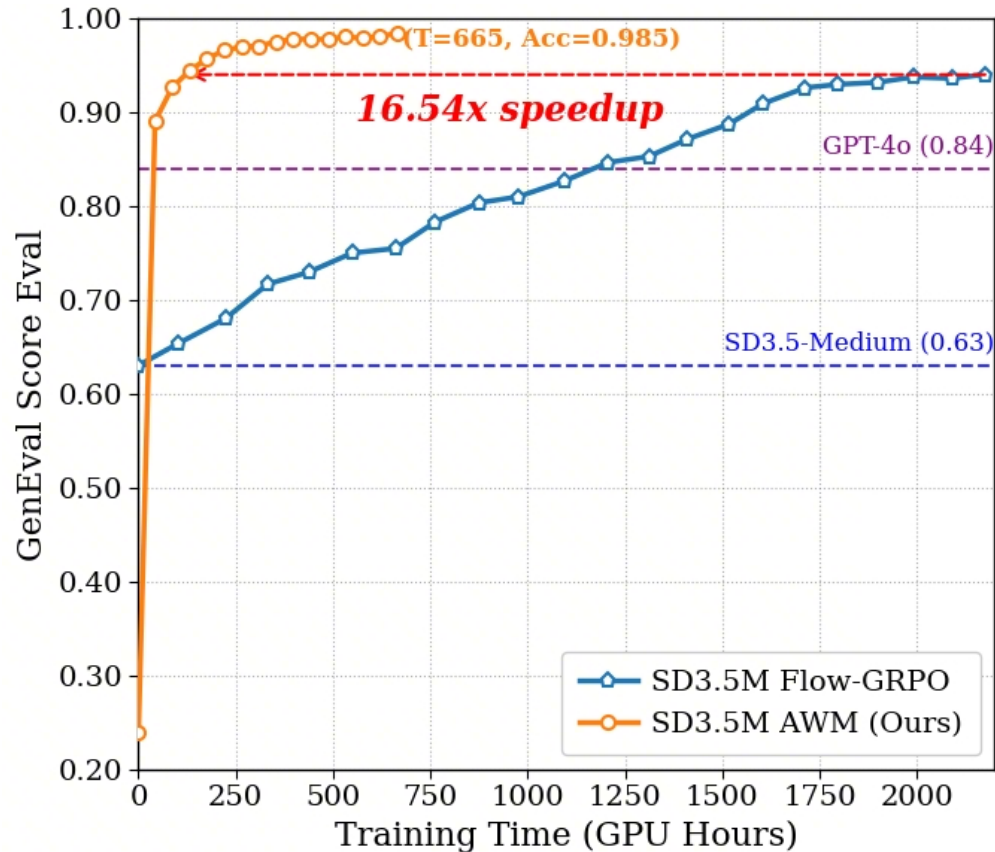
A zebra and a chair



A runway displaying
“Spring Collection 2024”



GenEval: reach the baseline score sooner



(a) Faster Convergence

16.54×

GPU-time speedup to the Flow-GRPO comparison score.

Final reported GenEval scores:

SD3.5M 0.630

Flow-GRPO 0.950

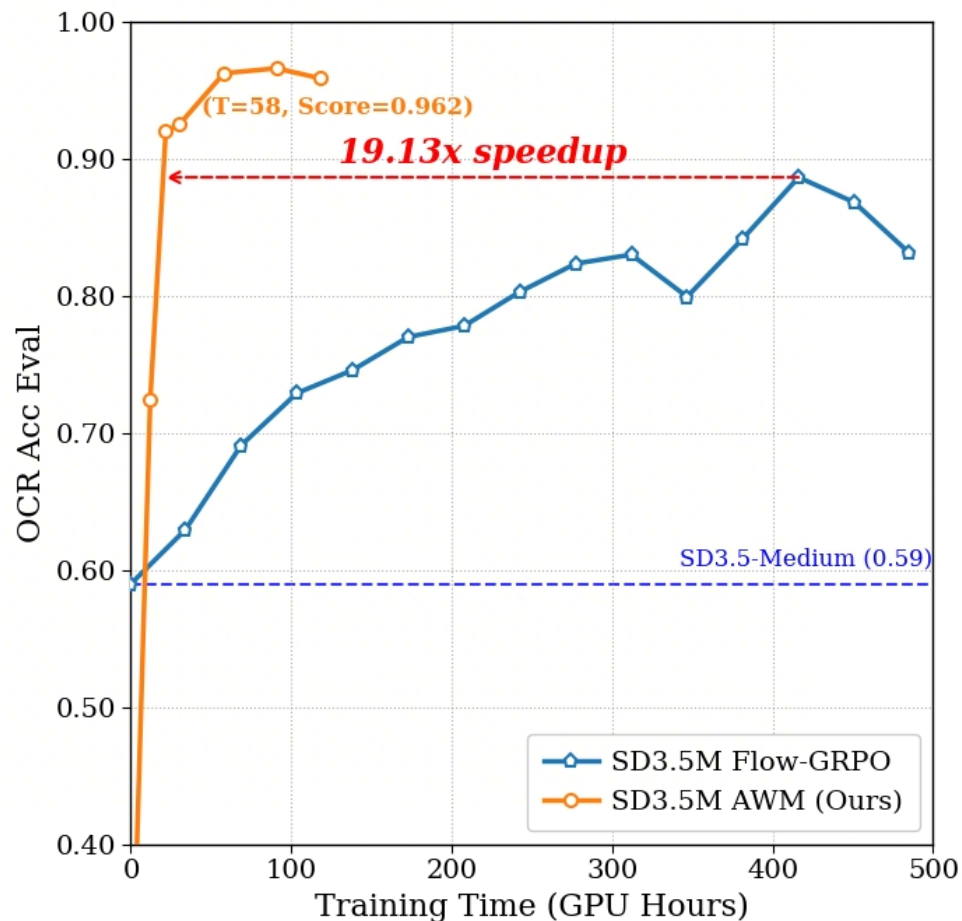
TempFlow-GRPO 0.970

AWM 0.985

Speedup and final score refer to different points on the learning curve.

Original curve: Xue et al., AWM (2026), Figure 1; scores: Table 1. End-to-end configurations differ; this is not an isolated estimator ablation.

OCR: faster learning of a verifiable reward



19.13×

GPU-time speedup at the reported comparison point.

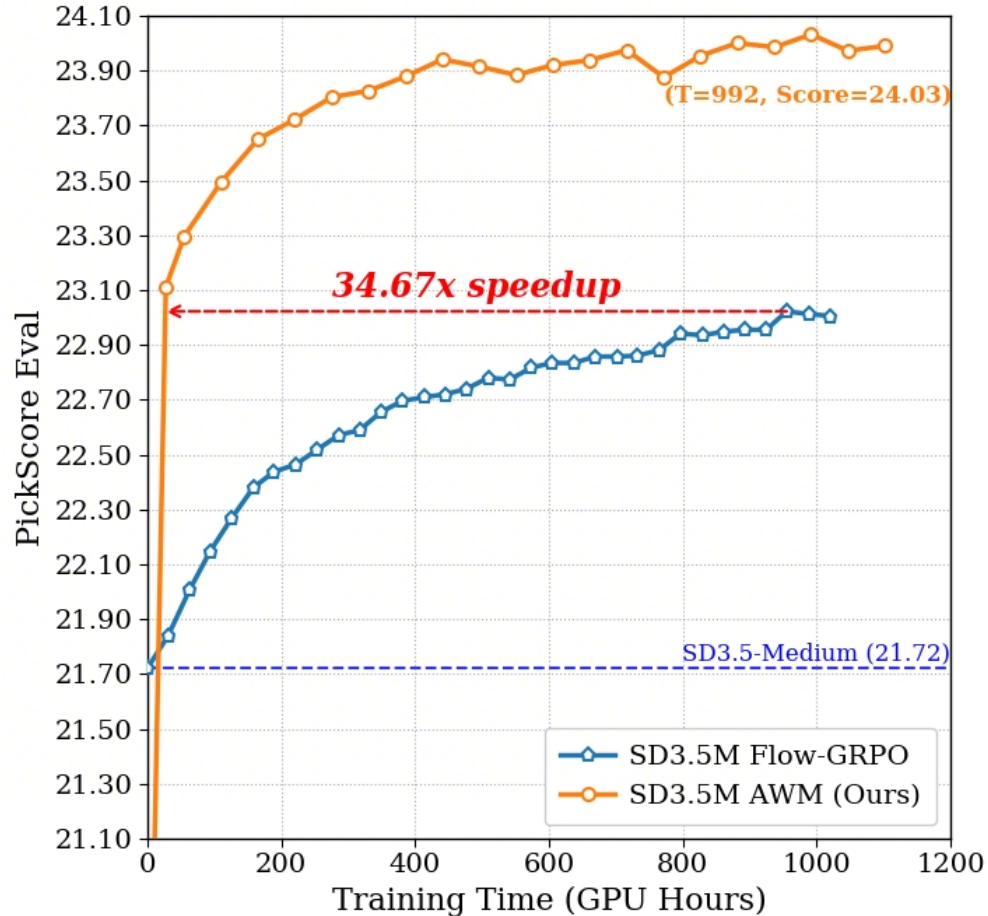
Flow-GRPO: **0.89** at 415.9 GPU h.

AWM: **0.92** at 21.76 GPU h.

The AWM curve later reaches about **0.96**.

Backbone: SD3.5M. The graph is reproduced from the paper.

PickScore: gains also extend to learned preferences



34.67×

GPU-time speedup at the reported comparison point.

Flow-GRPO: **23.01** at 956.1 GPU h.

AWM: **23.11** at 27.56 GPU h.

Later AWM score: **24.03**.

A learned reward gain is evidence about that proxy; independent human evaluation remains valuable.

SD3.5M, original curve. Xue et al., AWM (2026), Figure 5 and Appendix H.

Summary

1 **Reward feedback provides two routes to learning.**

Policy gradients use reward values; pathwise gradients differentiate through the reward and sampler.

2 **AWM connects reward optimization to diffusion pretraining.**

The ELBO gives an advantage-weighted matching surrogate. AWM learns faster on GenEval, OCR, and PickScore in our experiments.

3 **Time weighting controls the emphasis across noise levels.**

Likelihood weighting need not match perceptual importance; AWM shifts training emphasis toward noisier states.

Questions

Advantage Weighted Matching · arxiv.org/abs/2509.25050